Phase 1 — Transforms

Part B: homogeneous transforms in 2D

Robotics + AI Mentor Notes

August 20, 2026

Abstract

Goal: pack **rotation** + **translation** into a single matrix so combining frames becomes one clean multiplication. This is the exact tool ROS 2's TF tree and every robot arm use to chain frames. You already built `translate` and `rotate` (Part A). Now we fuse them.

Contents

1 The problem with keeping rotation and translation separate **1**

2 The trick: add a "1" to every point **2**

3 The homogeneous transform matrix (2D) **2**

4 Composing (chaining) frames = matrix multiplication **2**

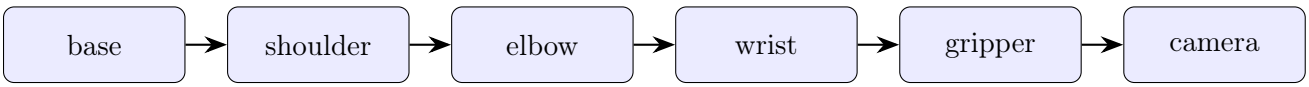
5 Your task (implement + verify) **3**

What's next **3**

---

1 The problem with keeping rotation and translation separate

A real camera is **both** rotated *and* shifted relative to the robot base. In Part A you'd handle that in two steps: first `rotate`, then `translate`. That works for one hop. But robots have **chains** of frames:



Doing rotate-then-add at every link, by hand, is error-prone and ugly. We want a single object per link that we can just **multiply together**. That object is the **homogeneous transform matrix**.

2 The trick: add a "1" to every point

We glue an extra `1` onto each 2D point, making it a 3-vector:

$$\mathbf{p} = \begin{bmatrix} x \\ y \end{bmatrix} \longrightarrow \tilde{\mathbf{p}} = \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

This is called **homogeneous coordinates**. That harmless-looking `1` is what lets a single matrix multiply do *both* rotation and translation at once.

3 The homogeneous transform matrix (2D)

A rotation by θ **and** a translation $[t_x, t_y]$ combine into one 3×3 matrix:

$$T = \begin{bmatrix} \cos\theta & -\sin\theta & t_x \\ \sin\theta & \cos\theta & t_y \\ 0 & 0 & 1 \end{bmatrix}$$

- **Top-left** 2×2 **block** = the rotation matrix $R(\theta)$ from Part A.
- **Top-right column** $[t_x, t_y]$ = the translation.
- **Bottom row** $[0, 0, 1]$ = bookkeeping that keeps the math consistent.

Applying it to a homogeneous point does rotation *then* translation in one shot:

$$\tilde{\mathbf{p}}' = T\,\tilde{\mathbf{p}} = \begin{bmatrix} \cos\theta\, x - \sin\theta\, y + t_x \\ \sin\theta\, x + \cos\theta\, y + t_y \\ 1 \end{bmatrix}$$

Then you **drop the last component** to get your 2D answer $[x', y']$.

Sanity checks (in your head):

- $\theta = 0,\ t = [1, 0]$ applied to $[0.4, 0.1] \to [1.4, 0.1]$ (pure translation, matches Part A).
- $\theta = 90°,\ t = [0, 0]$ applied to $[1, 0] \to [0, 1]$ (pure rotation, matches Part A).
- $\theta = 90°,\ t = [1, 0]$ applied to $[1, 0] \to$ rotate $[1, 0] \to [0, 1]$, then add $[1, 0] \to [1, 1]$.

4 Composing (chaining) frames = matrix multiplication

The real payoff: if $T_{A \leftarrow B}$ takes points from frame B into A, and $T_{B \leftarrow C}$ takes points from C into B, then getting from C **all the way to** A is just the matrix product:

$$T_{A \leftarrow C} = T_{A \leftarrow B}\, T_{B \leftarrow C}$$

One multiply per link, chained down the arm. **This is why homogeneous transforms exist.**

> Order matters: the matrix on the **right** is applied to the point **first**. $T_1 T_2 \mathbf{p}$ means "do T_2, then T_1."

5 Your task (implement + verify)

Open `src/utils/homogeneous_2d.py`. Implement:

1. `transform_matrix(theta, tx, ty)` — build the 3×3 matrix above.
2. `apply_transform(T, point)` — turn `point` into `[x, y, 1]`, multiply by `T`, return the 2D `[x', y']` (drop the trailing 1).
3. `compose(T1, T2)` — return the 3×3 matrix product `T1 @ T2` (by hand, no numpy).

Then run:

```
cd ~/Music/projects_abbas/ROBOT_VLA
python3 src/utils/homogeneous_2d.py
```

Expected output when correct:

```
[test] transform_matrix identity -> PASS
[test] pure translation          -> PASS
[test] pure rotation 90deg       -> PASS
[test] rotation + translation    -> PASS
[test] compose (chain frames)    -> PASS
All tests passed.
```

If a test fails: re-read the matrix in §3, check row/column order, fix, rerun. Report your output. Once green, we go to **Part C: 3D frames + the ROS 2 TF tree.**

What's next (do NOT start yet)

Part C: 3D homogeneous transforms (4×4) and how ROS 2 TF represents the whole robot as a tree of these.